



NVIDIA Stack (5 lapis)

- Hardware GPU di paling bawah; Tools di paling atas
- CUDA = jembatan agar GPU bisa komputasi umum
- Kita menulis di Framework, GPU yang kerja keras
- Tiap lapisan berdiri di atas lapisan bawahnya

► Memetakan posisi tiap komponen NVIDIA

Kenapa GPU (recap M02)

- CPU: sedikit core pintar, kerja berurutan
- GPU: ribuan core sederhana, kerja paralel
- Deep learning = tumpukan matrix operations
- Notebook hanya ukur waktu di GPU, bukan vs CPU

► Memahami kecocokan GPU untuk DL

Benchmark phi-2

- microsoft/phi-2 (~2.7B parameter)
- FP16 + device_map='auto' = pola load NVIDIA
- Warmup dulu, lalu generate() 5x, dirata-rata
- Benchmark = rata-rata waktu generate di GPU

```
AutoModelForCausalLM.from_pretrained("microsoft/p
hi-2", torch_dtype=torch.float16,
device_map="auto")
```

► Mengukur kecepatan inference di GPU

Half Precision FP16 vs FP32

- FP32: 32-bit, presisi penuh, boros memori
- FP16: 16-bit, hemat ~50% memori, cepat di Tensor Cores
- Notebook pakai FP16 PENUH (torch.float16)
- Bukan mixed precision/AMP (itu dibahas di M02)

```
torch_dtype=torch.float16
```

► Hemat memori GPU & inference lebih cepat

TensorRT (optimasi inference)

- Alat NVIDIA mempercepat inference (saat dipakai)
- Layer fusion + kernel auto-tuning
- Precision calibration ke FP16/INT8
- TF-TRT tipikal 2-5x lebih cepat (angka NVIDIA)

► Deploy model agar lebih cepat (di M06 hanya konseptual, tanpa demo)

NeMo (Neural Modules)

- Toolkit open-source NVIDIA untuk conversational AI
- Kaya pre-trained model siap pakai, tanpa training
- Modular seperti balok lego, dioptimalkan GPU
- Inti: unduh model, panggil method, langsung pakai

► Butuh task AI percakapan siap pakai

NeMo: NER, QA & Classification

- NER: TokenClassificationModel + ner_en_bert (BERT)
- NER tandai entitas: NVIDIA->ORG, New York->LOC
- QA: QAModel + qa_squadv1.1_bertbase (start-end jawaban)
- Classification: TextClassificationModel -> label kategori

```
TokenClassificationModel.from_pretrained("ner_en_
bert")
```

► Memahami & mengelompokkan teks

NeMo: Punctuation, MT & TTS

- Punctuation: PunctuationCapitalizationModel + punctuation_en_bert
- MT: MTEncDecModel + nmt_en_zh_transformer24x6 (EN->ZH)
- TTS tahap 1 FastPitch: teks -> spectrogram
- TTS tahap 2 HiFiGAN (vocoder): spectrogram -> audio

```
audio =
vocoder.convert_spectrogram_to_audio(spec=spec)
```

► Merapikan teks, terjemah, & sintesis suara

NVIDIA-Accelerated RAG

- Pipeline RAG sama (M05), yang ditukar = tooling NVIDIA
- Embedding all-MiniLM-L6-v2 & LLM gpt2 dipindah ke GPU (.to(self.device))
- Index FAISS IndexFlatL2; cudf (RAPIDS) bungkus dokumen -> pandas
- Catatan: MegatronGPT hanya di-import; IndexFlatL2 sebenarnya di CPU

```
df = cudf.DataFrame({'text':
documents}).to_pandas()
```

► Menjalankan RAG dengan akselerasi GPU

Quick-Ref: Model & Library per Task

Task	Model / Library	Catatan
Benchmark	microsoft/phi-2 (~2.7B)	FP16 + device_map='auto'
Optimasi inference	TensorRT (TF-TRT)	2-5x, konseptual (tanpa demo)
NER	TokenClassificationModel / ner_en_bert	berbasis BERT
Question Answering	QAModel / qa_squadv1.1_bertbase	prediksi start-end
Text Classification	TextClassificationModel	label per kalimat
Punctuation	PunctuationCapitalizationModel / punctuation_en_bert	rapikan transkrip
Machine Translation	MTEncDecModel / nmt_en_zh_transformer24x6	EN -> Mandarin
TTS	FastPitch + HiFiGAN	spectrogram lalu audio
RAG embedding	SentenceTransformer all-MiniLM-L6-v2	di GPU (.to(self.device))
RAG index / LLM	FAISS IndexFlatL2 / gpt2	faiss-gpu install, index di CPU

Istilah Kunci

CUDA — Platform NVIDIA yang menjembatani GPU untuk komputasi umum (general-purpose).

Tensor Cores — Unit GPU NVIDIA khusus yang sangat cepat untuk operasi FP16.

FP16 / FP32 — Angka 16-bit (hemat ~50% memori) vs 32-bit (presisi penuh, boros).

Mixed precision (AMP) — Campur FP32+FP16 saat training; berbeda dari FP16 penuh, dibahas di M02.

TensorRT — Library NVIDIA untuk optimasi inference via layer fusion & precision calibration.

NeMo — Toolkit open-source NVIDIA berbasis pre-trained model untuk conversational AI.

Vocoder (HiFiGAN) — Komponen TTS yang mengubah spectrogram menjadi audio.

cudf (RAPIDS) — DataFrame ber-GPU dari RAPIDS; di notebook hanya bungkus dokumen lalu .to_pandas().